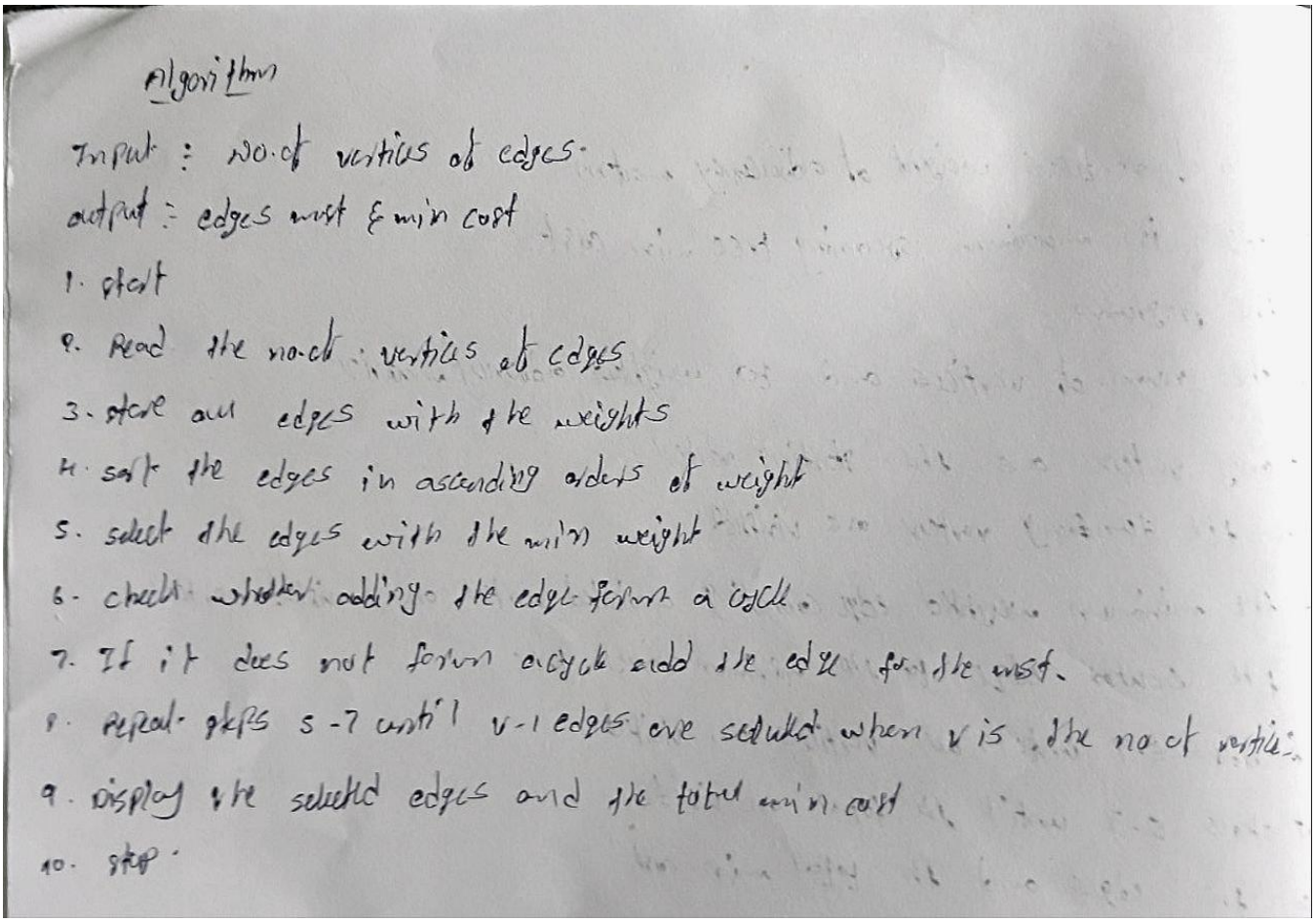
# PRACTICAL –10

**Aim:** Implement Kruskal's Algorithm

**Algorithm:**



Algorithm

Input : no. of vertices of edges.
output : edges must & min cost
1. start
2. Read the no. of vertices of edges
3. store all edges with the weights
4. sort the edges in ascending orders of weight
5. select the edges with the min weight
6. check whether adding the edge forms a cycle.
7. If it does not forms a cycle add the edge for the mst.
8. Repeat steps 5-7 until v-1 edges are selected when v is the no of vertices
9. display the selected edges and the total min cost
10. stop.

**Progarm:**

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Edge {
```

```
    int u, v, weight;
};

bool compare(Edge a, Edge b) {
    return a.weight < b.weight;
}

int findParent(int parent[], int i) {
    if (parent[i] == i)
        return i;
    return findParent(parent, parent[i]);
}

int main() {
    int n, e;
    Edge edges[100];

    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter number of edges: ";
    cin >> e;
```

```
cout << "Enter edges (source destination weight):\n";

for (int i = 0; i < e; i++) {
    cin >> edges[i].u >> edges[i].v
        >> edges[i].weight;
}

sort(edges, edges + e, compare);

int parent[100];

for (int i = 1; i <= n; i++) {
    parent[i] = i;
}

int minCost = 0;
int count = 0;

cout << "\nEdges in Minimum Spanning Tree:\n";

for (int i = 0; i < e && count < n - 1; i++) {
```

```
        int u = findParent(parent, edges[i].u);
        int v = findParent(parent, edges[i].v);

        if (u != v) {
            cout << edges[i].u << " - "
                 << edges[i].v << " : "
                 << edges[i].weight << endl;

            minCost += edges[i].weight;
            parent[u] = v;
            count++;
        }
    }

    if (count != n - 1) {
        cout << "Graph is disconnected.\n";
    } else {
        cout << "Minimum Cost = " << minCost << endl;
    }

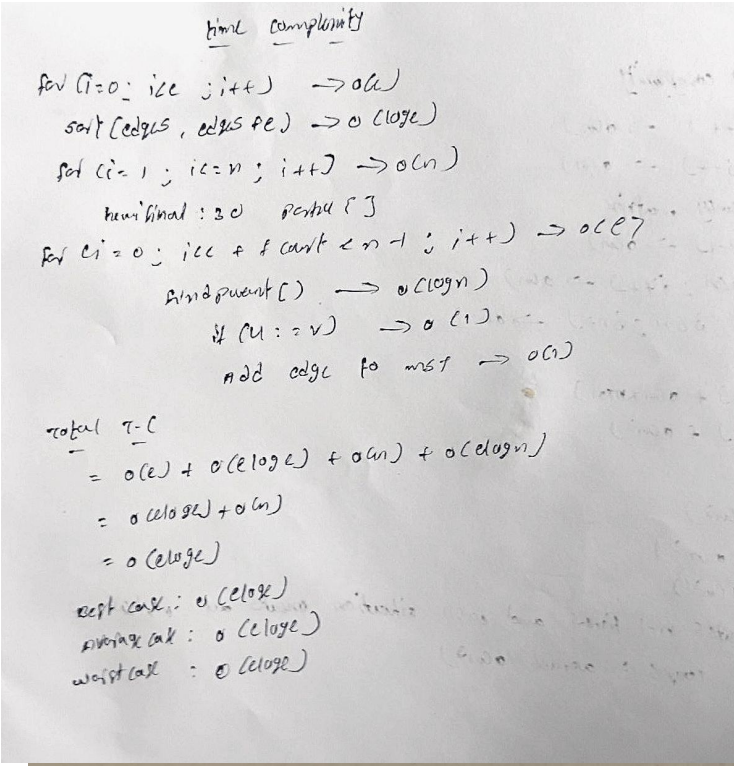
    return 0;
}
```

## Output:

```
Enter number of vertices: 4
Enter the adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 5
6 8 5 0

Edges in Minimum Spanning Tree:
1 - 2 : 2
2 - 3 : 3
3 - 4 : 5
Minimum Cost = 10
```

## Time complexity:




**Conclusion:** Kruskal's algorithm has a time complexity of (O(Elog E), mainly because it sorts all edges in ascending order of their weights.

Nam e: M.Hima Sankar
Enrollment no : 92460118218